CSCI3170
Introduction to Database Systems

7.5-functional_dependencies_inter

Slides: Michael Yu

Recap(1)

- *Due to version differences when formatting*
  - *When referring to the closer of functional dependencies or attribute(s), please treat F+ == F^+*

- Recall:
  - Armstrong's Axioms is sound and complete.
    - Sound: they generate only FDs in F^+.
    - Complete: repeated application of these rules will generate all FDs in F^+.
  - Set of all FDs implied by a given set F of FDs is called the closure of F, denoted as F^+.
  - Armstrong's Axioms, can be applied repeatedly to infer all FDs implied by a set of FDs.

Recap(2)

- Given the below relation
  - *student_ID* $\rightarrow$ *student_name*
  - *course_ID* $\rightarrow$ *course_name*
  - *course_name* $\rightarrow$ *department_name*
- By Armstrong's Axioms, these implied too (not exhaustive)
  - *student_ID, student_name* $\rightarrow$ *student_ID*
  - *student_ID, student_name* $\rightarrow$ *student_name*
  - *student_ID, course_name* $\rightarrow$ *student_name, course_name*
  - *course_ID* $\rightarrow$ *department_name*
  - ...

student_ID	student_name	course_ID	course_name	department_name
111	Chan	3170	DB	CSE
222	Wong	3170	DB	CSE
333	Tam	3160	Cal	MATH
111	Chan	3160	Cal	MATH

Recap(3) Check Understanding

- Given F = {A → B, A → C, BC → D}
- Prove A → D:

Cheat Sheet
F1 (Reflexivity) If $X \supseteq Y$ then $X \rightarrow Y$
F2 (Augmentation) $\{X \rightarrow Y\} \models XZ \rightarrow YZ$
F3 (Transitivity) $\{X \rightarrow Y, Y \rightarrow Z\} \models X \rightarrow Z$
F4 (Additivity) $\{X \rightarrow Y, X \rightarrow Z\} \models X \rightarrow YZ$
F5 (Projectivity) $\{X \rightarrow YZ\} \models X \rightarrow Y$
F6 (Pseudo-transitivity) $\{X \rightarrow Y, YZ \rightarrow W\} \models XZ \rightarrow W$

Continue: Armstrong's Axioms

- **Additional Rules** we inferred from Armstrong's axioms.
  - Rule 4 (**additivity**):
    - If $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta\ \gamma$ holds
  - Rule 5 (**projectivity**):
    - If $\alpha \rightarrow \beta\ \gamma$ holds, then $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds
  - Rule 6 (**pseudo-transitivity**):
    - If $\alpha \rightarrow \beta$ holds and $\gamma\ \beta \rightarrow \delta$ holds, then $\alpha\ \gamma \rightarrow \delta$ holds

- **Note:**
  - Additivity aka Union
  - Projectivity aka Decomposition

(Extended) Armstrong's Axioms

- *Deriving: projectivity*
- {X → Y Z} |= X → Y

Cheat Sheet

F1 (Reflexivity) If $X \supseteq Y$ then $X \rightarrow Y$.

F2 (Augmentation) $\{X \rightarrow Y\} \models XZ \rightarrow YZ$.

F3 (Transitivity) $\{X \rightarrow Y, Y \rightarrow Z\} \models X \rightarrow Z$.

Symbol |= denotes infers, i.e., FDs on LHS infer the FDs on RHS

(Extended) Armstrong's Axioms

- *Deriving: Pseudo-transitivity*
- {X → Y , Y Z → W} |= XZ → W

Cheat Sheet

F1 (Reflexivity) If $X \supseteq Y$ then $X \rightarrow Y$.

F2 (Augmentation) $\{X \rightarrow Y\} \models XZ \rightarrow YZ$.

F3 (Transitivity) $\{X \rightarrow Y, Y \rightarrow Z\} \models X \rightarrow Z$.

(Extended) Armstrong's Axioms

- *Deriving: <u>Additivity</u>*
- {X→ Y , X → Z} |= X → Y Z

Cheat Sheet
F1 (Reflexivity) If $X \supseteq Y$ then $X \rightarrow Y$.
F2 (Augmentation) $\{X \rightarrow Y\} \models XZ \rightarrow YZ$.
F3 (Transitivity) $\{X \rightarrow Y, Y \rightarrow Z\} \models X \rightarrow Z$.

Final Points on Closures

- Definition: set of all dependencies that can be inferred from F is called the **closure** of F
  - F denotes set of FD's on relational schema R
  - F^+ denotes the closure of F
  - F^+ includes dependencies in F
  - F^+ is **closed** under Armstrong's axioms

Motivating Example

- Given
  - F = { X → Y, Y → Z}
- Can X → Z be inferred/derived from FDs in F?

- Approach One:
  - check for X → Z through checking membership in F+ (by computing F+) ?
  - Basically…
    - F+ = {XY → X, XY → Y, XY → Z, XZ → X, XZ → Y, XZ → Z, XYZ → X, XYZ → Y, XYZ → Z, XY → XY, XY → YZ, XY → XZ, … , X → Z , … }
    - Based on F+, X → Z is in the closure of F.

Finding F+

- $F^+ = F$

  repeat

  for each FD f in F^+

  apply reflexivity and augmentation rules on f

  add resulting FD(s) to F^+

  for each pair of FDs f_1 and f_2 in F^+

  if f_1 and f_2 can be combined using transitivity

  then add resulting FD(s) to F^+

  until F^+ does not change any further

Finding F+

- $A \rightarrow A$, $B \rightarrow B$, $C \rightarrow C$,
- $AB \rightarrow AB$, $BC \rightarrow BC$, $AC \rightarrow AC$, $ABC \rightarrow ABC$,
- $AB \rightarrow A$, $AB \rightarrow B$,
- $BC \rightarrow B$, $BC \rightarrow C$,
- $AC \rightarrow A$, $AC \rightarrow C$,
- $ABC \rightarrow AB$, $ABC \rightarrow BC$, $ABC \rightarrow AC$,
- $ABC \rightarrow A$, $ABC \rightarrow B$, $ABC \rightarrow C$,
- $A \rightarrow B$, … (1) (given) $B \rightarrow C$, … (2) (given)
- $A \rightarrow C$, … (3) (transitivity on (1) and (2))
- $AC \rightarrow BC$, … (4) (augmentation on (1))
- $AC \rightarrow B$, … (5) (decomposition on (4))
- $A \rightarrow AB$, … (6) (augmentation on (1))
- $AB \rightarrow AC$, $AB \rightarrow C$, $B \rightarrow BC$,
- $A \rightarrow AC$, $AB \rightarrow BC$, $AB \rightarrow ABC$, $AC \rightarrow ABC$, $A \rightarrow BC$, $A \rightarrow ABC$

<terminate from successful completion>

$F = (A, B, C)$

$F = \{ A \rightarrow B, B \rightarrow C \}$

Check Attribute Closure instead of F+

- Computing closure of a set of FDs ~~can~~ be **is** expensive

- In many cases, we just want to check if a given FD, $X \rightarrow Y$, is in F^+.
  - So for when checking for X → Z, given F = { X → Y, Y → Z}
    1. Compute X^+ instead of F^+
    2. We then check if Z is covered by X^+
  - *Where…*
    - *X and Z* - a set of attributes
    - *F* - a set of functional dependencies on R

- Definition: Given a set of attributes a, define the ***closure*** of A **under** *F* (denoted by A^+) as the set of attributes that are functionally determined by a under *F.*

Computing Attribute Closures

- **Pseudocode** to the closure of A under *F*
  - *result* := A;
    while (changes to *result*) **do**
    for each $\beta \rightarrow \gamma$ **in** *F* **do**
    begin
    if $\beta \subseteq$ *result* **then** *result* := *result* $\cup \gamma$
    end

- When no additional changes to result is possible, the final value of variable result is A^+
  - $F = \{ A \rightarrow B, B \rightarrow C \}$
    - A^+ = ?
    - B^+ = ?
    - C^+ = ?
    - AB^+ = ?

Practice A

- R = (A, B, C, G, H, I)
- F = {A $\rightarrow$ B, A $\rightarrow$ C, CG $\rightarrow$ H, CG $\rightarrow$ I, B $\rightarrow$ H}
- **Task: Compute the closure of AG**

```
result := A;
while (changes to result) do
  for each β → γ in F do
    begin
      if β ⊆ result
        then
              result := result ∪ γ
      end
```

Try it yourself

- F = { A → B, BC → D, A → C }
- Practice: **Compute A+**

```
result := A;
while (changes to result) do
  for each β → γ in F do
    begin
      if β ⊆ result
        then
              result := result ∪ γ
      end
```

FD(s) applied: Key(s)

- FDs are a **generalization** of a concept of key(s)
  - Key are a specialization of functional dependencies

- Consider
  - schema *STUDENT(zid, name, address)*
  - Given *zid* $\rightarrow$ *name, address*

- Notes:
  - For any tuple, value of the key of a relation always functionally determine every remaining attribute in that relation
  - Left-hand side of a dependency does not imply uniqueness

FD(s) applied: Key(s)

Note: γ and α each refer to a set of attributes

- Given
  - *student_ID* $\rightarrow$ *student_name*
  - *course_ID* $\rightarrow$ *course_name*
- Check understanding:
  - *(student_ID, course_ID)* candidate key?
  - *(student_ID, course_ID, course_name)* not candidate key?

student_ID	student_name	course_ID	course_name
111	Chan Tai Man	3170	Database
222	Wong Siu Ling	3170	Database
333	Tam Wai Ming	3160	Algorithms
111	Chan Tai Man	3160	Algorithms

α is a super key for R (minimal/non-minimal)
iff $\alpha \rightarrow R$
where *R* is schema for a relation R
α is a candidate key for R iff
$\alpha \rightarrow R$, and
for no γ that is a proper subset of α, $\gamma \rightarrow R$ (minimal property).

FD(s) applied: Key(s)

- Example1:
  - *R = (A, B, C, G, H, I)*
  - *F = {A → B, A → C, CG → H, CG → I, B → H}*
  - *Let $(AG)^+$ = ABCGHI, does this mean AG is candidate key?*

- Example2:
  - *inst_dept (ID, dept_name, name, salary, building, budget)*
  - *dept_name → building*
  - *ID → building*
  - *Comment on a possible primary key (chosen candidate key)? Might there be many options?*

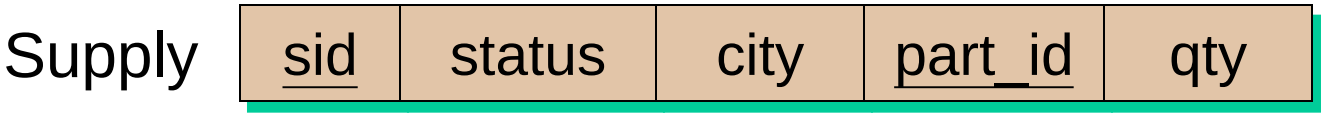
FD(s) applied: Normal Form(s)

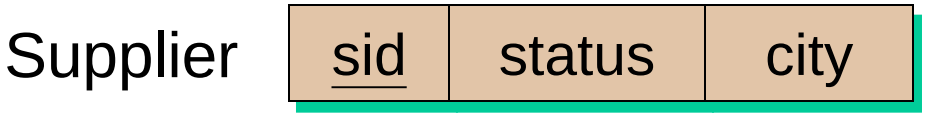
• 1st Normal Form	• No repeating data groups
• 2nd Normal Form	• No partial key dependency
• 3rd Normal Form	• No transitive dependency
• Boyce-Codd Normal Form	• Reduce keys dependency
• 4th Normal Form	• No multi-valued dependency
• 5th Normal Form	• No join dependency

$$1NF \supset 2NF \supset 3NF \supset BCNF \supset 4NF \supset 5NF$$

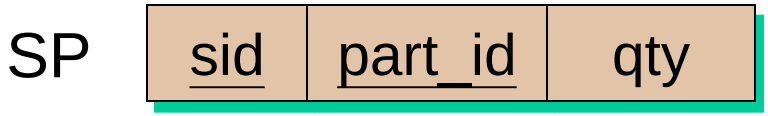
DB Normalization via Decomposition

- A set of relation schemas { R_1, R_2, …, R_n }, with n ≥ 2 is a **decomposition** of R if $R_1 \cup R_2 \cup \ldots \cup R_n = R$



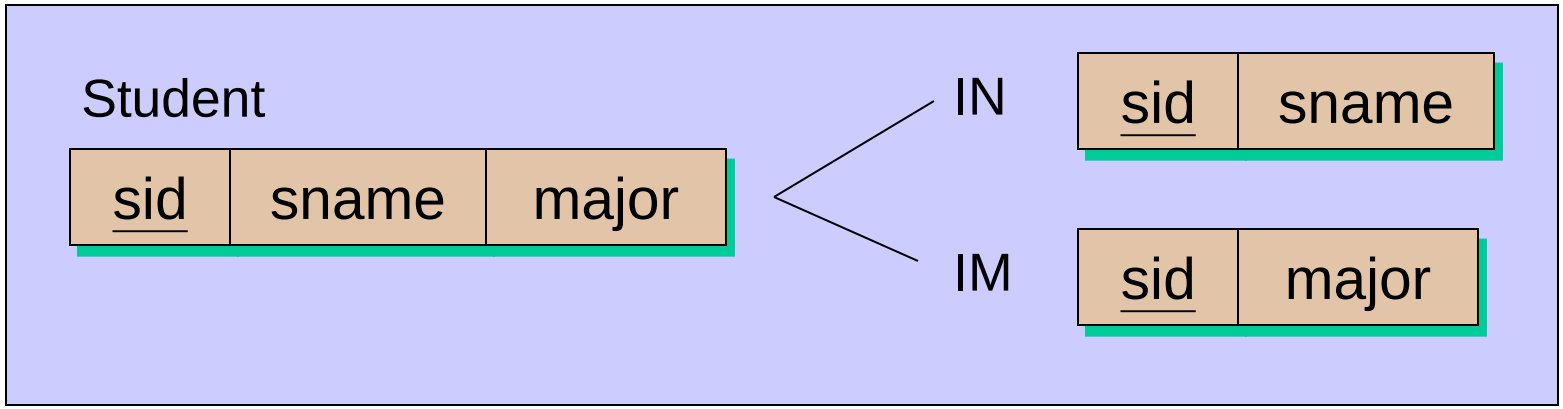


and



Property One: Decomposition Lossless

- Relational schemas { R_1, R_2, …, R_n } is a **lossless-join decomposition** of R if For all possible relation instances *r* on schema *R*…
  - $r = \Pi_{R1}(r) \bowtie \Pi_{R2}(r) \bowtie \ldots \bowtie \Pi_{Rn}(r)$



Student

sid	sname	major
123	Ling Wang	C.S.
456	Hong Zhou	C.S.

IN

sid	sname
123	Ling Wang
456	Hong Zhou

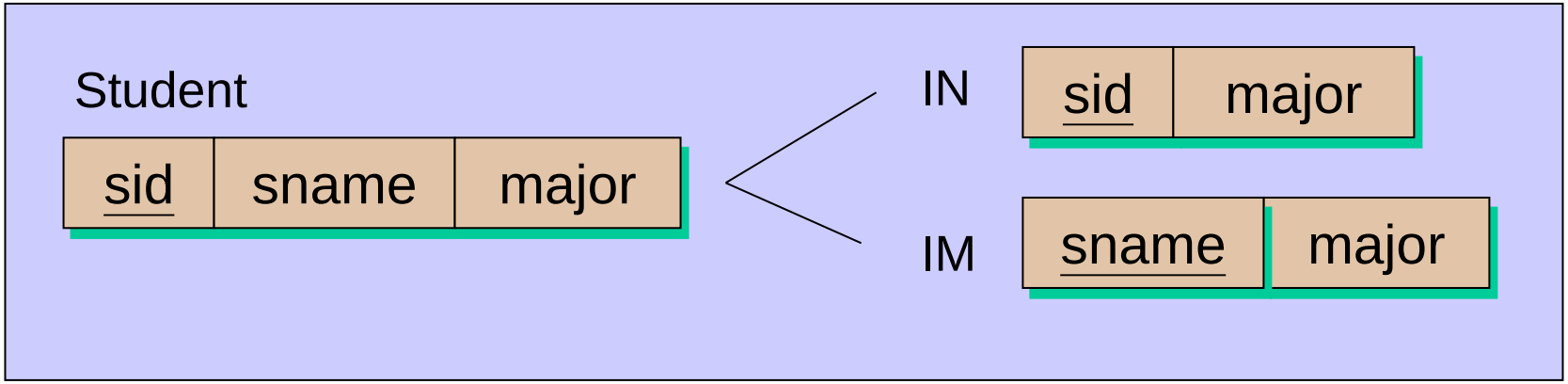
IM

sid	major
123	C.S.
456	C.S.

Can 'Student' can be recovered by joining instances of IN and IM

Property One: Lossless

- Here the decomposition is lossy, which is problematic



Student

sid	sname	major
123	Ling Wang	C.S.
456	Hong Zhou	C.S.

IN

sid	major
123	C.S.
456	C.S.

IM

sname	major
Ling Wang	C.S.
Hong Zhou	C.S.

Can 'Student' can be recovered by joining instances of IN and IM

Decomposition Revisited

- In second decomposition…
  - Original instance of 'Student' can never be recovered
    - Through joining instances of IN and IM
  - Therefore decomposition is **NOT** lossless

IN

sid	major
123	C.S.
456	C.S.

IM

sname	major
Ling Wang	C.S.
Hong Zhou	C.S.

IN ⋈ IM

sid	sname	major
123	Ling Wang	C.S.
123	Hong Zhou	C.S.
456	Ling Wang	C.S.
456	Hong Zhou	C.S.

≠

Student

sid	sname	major
123	Ling Wang	C.S.
456	Hong Zhou	C.S.

Property One: Lossless

- *Given,* relation schema R, and set F of FDs on *R*
- **Theorem:**
  - **Decomposition of R into relations with attribute sets R_1, R_2 is a lossless-join decomposition** iff
    - $(R_1 \cap R_2) \rightarrow R_1 \in F^+$ OR $(R_1 \cap R_2) \rightarrow R_2 \in F^+$
  - i.e., $R_1 \cap R_2$ is a **superkey** for R_1 or R_2

- *Note, this for cases where decomposition R into only R1, R2*

Check your understanding (1)

- To be lossless, when decomposing R to R_1 or R_2 , attributes common to R_1 and R_2 must contain a key for either R_1 or R_2 .

- Example
  - R = (A, B, C)
  - F = { A $\rightarrow$ B }
  - Decompose R into R1{ A, B }, R2{ A, C }
    - lossless join decomposition?
  - Decompose R into R1{ A, B }, R2{ B, C }
    - not lossless join decomposition?

Check your understanding (2)

- To be lossless, when decomposing R to R_1 or R_2 , attributes common to R_1 and R_2 must contain a key for either R_1 or R_2 .
- Example
  - $R = \{ A, B, C, D \}$
  - $F = \{ A \rightarrow B, C \rightarrow D \}$.
- Consider a two step decomposition:
  - Decompose R into $R_1 = (A, B), R_2 = (A, C, D)$
  - Decompose R_2 into $R_3 = (C, D), R_4 = (A, C)$
  - Final Decomposition of R: $\{ (A, B), (C, D), (A, C) \}$
  - a lossless join decomposition.
- *If* R is decomposed into $(A, B), (C, D)$
  - lossy-join decomposition.

Property 2: Dependency Preservation

- R - relation schema
- F - set of functional dependencies on R
- $\{ R_1, R_2 \}$ - decomposition of R.
- F_i - set of dependencies in F^+ involves only attributes in R_i.
  - F_i is called **projection** of F on the set of attributes of R_i.

- Theorem: dependency is preserved if…
  - Intuitively, a dependency-preserving decomposition allows us to enforce all FDs by examining a single relation instance on each insertion or modification of a tuple

$$(F_1 \cup F_2)^+ = F^+$$

Lossless Decomposition (1) (Dependency Not Preserved)

F =
{ sid → dname,
dname → dhead }

Student

sid	dname	dhead
123	C.S.	Ying Wang
456	C.S.	Ying Wang

IN

sid	dname
123	C.S.
456	C.S.

and

IH

sid	dhead
123	Ying Wang
456	Ying Wang

Update

sid	dhead
123	Ying Wang
456	Lin Cheung

Consider the RHS update.

This update violates the FD ‘dname → dhead’. However, it can sometimes only be caught when we join IN and IH

Case Study: Cause of Issue

- This decomposition similarly does not preserve dependency:
  - F = { sid $\rightarrow$ dname, dname $\rightarrow$ dhead }



- Projected dependancies
  - F_{IN} = { trivial dependencies, sid $\rightarrow$ dname, sid $\rightarrow$ sid dname}
  - F_{IH} = { trivial dependencies, sid $\rightarrow$ dhead, sid $\rightarrow$ sid dhead }

- Example of dependency not preserved/ enforced
  - dname $\rightarrow$ dhead $\in F^+$
  - dname $\rightarrow$ dhead $\notin (F_{IN} \cup F_{IH})^+$

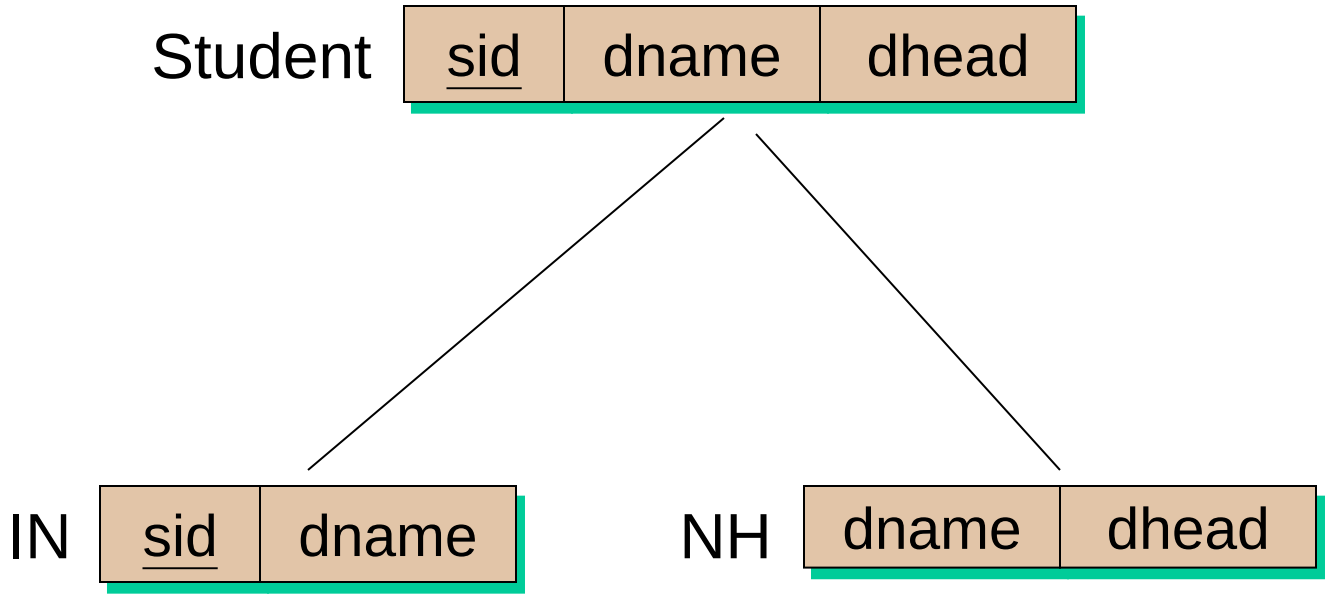
Theorem: Recall dependency is preserved if…

$$(F_1 \cup F_2)^+ = F^+$$

Case Study: Decomposition 2

- Dependency set:
  - F = { sid $\rightarrow$ dname, dname $\rightarrow$ dhead }

- If decomposed the relation in **another** way.



Case Study:

- F = { sid → dname, dname → dhead }

IN

sid	dname

NH

dname	dhead

- This decomposition preserves dependency:
  - F_{IN} = { trivial dependencies, sid → dname, sid → sid dname}
  - F_{NH} = { trivial dependencies, dname → dhead, dname → dname dhead }

- In fact…
  - $(F_{IN} \cup F_{NH})^+ = F^+$

- *Note: this decomposition is not lossless…*

Lossless AND Dependency Preserved

Student

sid	dname	dhead
123	C.S.	Ying Wang
456	C.S.	Ying Wang

IN

sid	dname
123	C.S.
456	C.S.

and

NH

dname	dhead
C.S.	Ying Wang
C.S.	Ying Wang

Update

dname	dhead
C.S.	Ying Wang
C.S.	Lin Cheung

The error in NH will immediately be caught by the DBMS, since it violates F.D. dname → dhead. No join is necessary.

Solution

To show correctness of the Projectivity rule:

if $X \rightarrow YZ$, then $X \rightarrow Y$ *(*and $X \rightarrow Z$*)* (**projectivity**)

Proof:

$X \rightarrow YZ$	… (1) (given)
$YZ \rightarrow Y$	… (2) (reflexivity)
$X \rightarrow Y$	… (3) (transitivity on (1), (2))
$YZ \rightarrow Z$	… (4) (reflexivity)
$X \rightarrow Z$	… (5) (transitivity on (1), (4))

Solution

To show correctness of the *Additivity* rule:

$X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$ (***Additivity***)

Proof:

$X \rightarrow Y$	… (1) (given)
$X \rightarrow Z$	… (2) (given)
$XX \rightarrow XY$	… (3) (augmentation on (1))
$X \rightarrow XY$	… (4) (simplify (3))
$XY \rightarrow ZY$	… (5) (augmentation on (2))
$X \rightarrow ZY$	… (6) (transitivity on (4) and (5))

Solution

To compute AG^+

$result = AG$

$result = ABG \quad (A \rightarrow B)$

$result = ABCG \quad (A \rightarrow C)$

$result = ABCGH \quad (CG \rightarrow H)$

$result = ABCGHI \quad (CG \rightarrow I)$